SwitchZip: A Cost-Aware Hybrid Classical/Neural Lossless Compressor

Final Year BS Project Proposal

Is it lossless?

Yes. SwitchZip only decides *which* lossless compressor to use on each block of data. Both options (classical and neural) are lossless on their own, and the choice made per block is stored alongside the data, so decompression always recovers the original file bit-for-bit.

1 Key Terms

Lossless compression
: Shrinking a file so it takes less space while still being able to reconstruct the exact original afterward. (Lossy compression, e.g. JPEG or MP3, permanently discards some detail.)

Compression ratio
: Original size ÷ compressed size. Example: a 10 MB file compressed to 2.5 MB has a ratio of 4.

Entropy coding
: Assigning shorter codes to more predictable data and longer codes to less predictable data. Huffman coding and arithmetic coding are both entropy coders.

Arithmetic coding
: Represents a whole message as one number in $[0, 1)$. At each step the current interval is split according to the predicted probability of each possible next symbol, and the piece matching the actual symbol becomes the new interval.

Predictor
: Anything that estimates "how likely is each possible next symbol, given what came before." Better predictions shrink the interval less on the correct guess, which is what makes the output smaller.

Neural predictor
: A predictor built from a neural network (e.g. an LSTM or a small transformer). Usually predicts far better than simple statistical models, at the cost of more computation per symbol.

Classical compressor
: A non-neural compressor such as `gzip`, `zstd`, or PPMd. Cheap and fast, weaker predictor.

Block
: A chunk of the file (e.g. 4–64 KB) compressed as one unit, so a program can make a separate decision for each chunk.

Gatekeeper
: The classifier this project builds: for each block, it predicts whether the neural predictor's gain is worth its extra time.

Latency
: How long compression/decompression takes to run.

2 Worked Example: Arithmetic Coding

Suppose the alphabet is {A, B, C} with predicted probabilities $P(A) = 0.7$, $P(B) = 0.2$, $P(C) = 0.1$, and the message to encode is "A".

Start with the interval $[0, 1)$. Split it into three pieces sized by probability:

$$A \to [0, 0.7), \quad B \to [0.7, 0.9), \quad C \to [0.9, 1.0)$$

Since the symbol is A, the new interval becomes $[0, 0.7)$ — a large range, so it costs few bits to identify a number inside it. If instead the predictor had assigned A only $P(A) = 0.1$, the interval for A would have been small (e.g. $[0, 0.1)$), costing more bits for the same symbol. This is exactly why a better predictor produces smaller output: it makes the interval for the symbol that actually occurs as large as possible.

3 Worked Example: Routing a File

Take a log file made of two halves: the first half is highly repetitive boilerplate (timestamps, fixed field names), the second half is free-form user comments.

- Block 1–50 (boilerplate): `gzip` already compresses this well because the patterns are short and repeat constantly. A neural predictor would barely improve the ratio, so the gatekeeper routes these blocks to the classical compressor.
- Block 51–100 (free text): predicting the next word needs real language understanding. `gzip` does much worse here, while a small neural model captures word-level structure and compresses it further. The gatekeeper routes these blocks to the neural compressor.

The output file stores: (compressed block 1–50 via gzip) + (compressed block 51–100 via neural model) + (a short list saying "blocks 1–50: classical, blocks 51–100: neural"). Decompression reads that list first and applies the matching decoder to each block.

4 Motivation

Neural predictors combined with arithmetic coding beat classical compressors by a wide margin on many data types [1, 2, 3]. The cost is speed: a neural model has to produce a prediction for every single symbol, and this speed/ratio tradeoff is identified as the main obstacle to using neural compression in practice [4]. There is also no standard way yet to decide, block by block, whether paying that cost is worth it [5].

5 Project Plan

1. **Baselines.** Run `gzip`, `zstd`, and a small open-source neural compressor (e.g. DZip [3]) on 3–4 public datasets: plain text, genomic data, log files.
2. **Labels.** For every block, record which method gave the best ratio-for-time result. This becomes training data for the gatekeeper.
3. **Gatekeeper.** Train a small, fast classifier (simple block statistics, or a tiny CNN) to predict that label directly from the raw block, without running the neural model first.
4. **Pipeline.** Block splitting → gatekeeper → route → compress → measure ratio and wall-clock time end to end.
5. **Ablations.** Vary block size; measure how often the gatekeeper is wrong and what that costs; test on data types not seen during training.

6 Evaluation

On held-out data, report: compression ratio vs. gzip, zstd, and a fully-neural baseline; total wall-clock time vs. the same baselines; gatekeeper accuracy; and a ratio-vs-time plot showing where SwitchZip sits relative to "always classical" and "always neural."

7 Extension: More Than Two Candidates

The gatekeeper does not have to be a two-way choice. It can be extended to pick among K candidate compressors per block — e.g. run-length encoding (RLE, good for long repeated runs), dictionary/LZ-style coding (good for repeated substrings), delta encoding (good for numeric or sensor data with small changes between values), and the neural predictor (good for complex structure).

This does not change the core mechanism: the gatekeeper is still a classifier making one cheap prediction per block, so the decision cost does not grow much with K. Two things do grow with K: the bookkeeping per block (a 2-way choice needs 1 bit, an 8-way choice needs 3 bits), and the amount of labeled training data needed so the gatekeeper sees enough examples of each candidate's "home turf." Recommended order of work: build and validate the two-way (classical vs. neural) version first, since it is easier to measure and debug, then generalize to K candidates as a stretch goal if time permits.

8 Novelty Check — Do This Before Starting

Before committing months to this project, search specifically for:

- Data Compression Conference (DCC) proceedings, last 3 years, for "adaptive" or "learned" codec selection.
- "Compressor routing," "codec selection," "compression algorithm selection," and "mixture of compressors."
- OpenZL (a graph-based compression framework from Meta that selects different encoding components per data structure) — close enough in spirit that the exact difference from SwitchZip needs to be stated clearly if this project proceeds.
- Context-mixing compressors such as PAQ, cmix, and zpaq — these *blend* predictions from multiple models continuously rather than *switching* between them per block. SwitchZip's discrete, cost-aware switching is the point of difference to state explicitly.

If a very similar cost-aware discrete router already exists in the literature, the project should pivot to whichever gap remains open — e.g. the K-way extension above, or a different data domain (genomic, sensor/IoT) where no one has applied this yet.

References

References

[1] G. Delétang, A. Ruoss, P.-A. Duquenne, E. Catt, T. Genewein, C. Mattern, J. Grau-Moya, L. K. Wenliang, M. Aitchison, L. Orseau, M. Hutter, and J. Veness. Language Modeling Is Compression. *International Conference on Learning Representations (ICLR)*, 2024.

[2] F. Bellard. Lossless Data Compression with Neural Networks. Technical report, 2019.

[3] M. Goyal, K. Tatwawadi, S. Chandak, and I. Ochoa. DZip: Improved General-Purpose Lossless Compression Based on Novel Neural Network Modeling. *arXiv preprint arXiv:1911.03572*, 2019.

[4] Kipf et al. Waiting to Decompress: The Economics of LLM-Based Compression. *Conference on Innovative Data Systems Research (CIDR)*, 2026.

[5] The 2026 Algorithmic Information Theory Data Compression Challenge. Benchmark report, 2026. `https://aitdcc.github.io`.